StoryFrames Backend: Step-by-Step Connection Plan

Frontend (Vite + React) ↔ Socket.IO ↔ FastAPI ↔ PostgreSQL. You write the code and the schema. This document says what to do, in what order, and how to know a step is finished.

How to use this plan

- Work in order. Do not start a step until the previous "Done when" check passes.
- Every step ends with a **Done when** check. If it fails, fix it before moving on.
- Keep the frontend running while you work. Each socket step ends with a real test in the browser.

Current setup (from your files)

Piece	What it is today	Address
Frontend	Vite 8, React 19, socket.io-client 4.8 (not connected yet)	http://localhost:5173
Backend	FastAPI with python-socketio 5.17 mounted as an ASGI app	http://localhost:8000
Database	PostgreSQL 17 in Docker (compose service `db`)	localhost:5432
ORM / migrations	SQLAlchemy 2.0.54, Alembic 1.20.0 (no models yet)	—

Compatibility: socket.io-client 4.x and python-socketio 5.x speak the same protocol and connect without extra setup. Do not change either side's major version during this plan.

Contents

1. Part 1: Environment and configuration
2. Part 2: Backend boots and answers over HTTP and Socket.IO
3. Part 3: Database connection and migrations
4. Part 4: Frontend connects to backend (first socket handshake)
5. Part 5: Schema and architecture decisions (your design work)
6. Part 5 (continued): Team and match model (step 5.6)
7. Part 6: Socket events, one at a time
8. Part 7: Replace the frontend's local stand-in with the real socket client
9. Part 8: AI story and scene generation
10. Part 9: Guessing, reveal and scoring
11. Part 10: Round switching and match end
12. Part 11: Later phases
13. Appendix A: Issues found in the current files
14. Appendix B: Checkpoint summary

Part 1: Environment and configuration

Goal: the backend and database start reliably on your machine, with one source of truth for the database URL.

Step 1.1: Decide where each piece runs

- **Recommended for development:** PostgreSQL in Docker, backend in a local virtual environment (`venv`). Reloads are faster and debugging is easier.
- Keep the `backend` service in `docker-compose.yml` for later, when you deploy.

DONE WHEN
You can say, in one sentence each, where the backend, the database, and the frontend run.

Step 1.2: Clean up the environment files

- Confirm `.env` is in `.gitignore`, so secrets are never committed.
- Keep `.env.example` as the template: every variable name with a placeholder value, no real secret.
- Make sure every variable the code reads appears in `.env.example`.

DONE WHEN
`git status` shows `.env` as ignored, and a new developer could copy `.env.example` and fill it in.

Step 1.3: Give the database URL two correct forms

- From your Windows machine (backend in venv): host `localhost`, port `5432`.
- From inside Docker (backend container): host `db` (the compose service name), port `5432`. Inside a container, `localhost` means the container itself, so it fails.
- Use the driver that matches the installed package. `psycopg2-binary` is in `requirements.txt`, so the scheme is `postgresql+psycopg2://`.
- Write both forms in `.env.example` as comments so they are not mixed up.

DONE WHEN
You can state which URL to use in each place, and it matches the driver in `requirements.txt`.

Step 1.4: Pass the database URL into the backend container

- The `backend` service in `docker-compose.yml` has no environment variables. Give it the container form of the URL, using an `environment` entry or an `env_file` entry.
- Keep the password in one place and reference it, rather than writing it twice.
- `POSTGRES_PASSWORD` on the `db` service must match the password in the URL.

DONE WHEN
`docker compose config` shows the `DATABASE_URL` value on the `backend` service.

Step 1.5: Make the settings loader independent of the working folder

- `app/db/database.py` reads `.env` relative to the current folder, so starting from another folder can fail.
- Point the settings loader at the backend root, not the current directory.
- Move the settings class to the current pydantic-settings style (`model_config`) to remove the deprecation warning from `class Config`.
- Make a missing `DATABASE_URL` produce a readable error, not a stack trace from deep inside SQLAlchemy.

DONE WHEN
Starting from the backend root and from one folder up gives the same settings, and a missing `DATABASE_URL` gives a clear message.

Step 1.6: Make Python and the dependencies match Docker

- The Dockerfile uses Python 3.12. Create the venv with Python 3.12 so local and container behavior match.
- `requirements.txt` looks like it is saved as UTF-16. Re-save it as UTF-8 (Notepad: *Save as*, encoding UTF-8).
- Create a fresh venv, install from `requirements.txt`, and confirm there are no errors.
- Keep the `venv` folder out of git (check `.gitignore`).

DONE WHEN
A brand-new venv installs cleanly, `python --version` shows 3.12.x, and `pip freeze` matches the pins.

Step 1.7: Start the database and check it from the host

- Run `docker compose up -d db`.
- If a local PostgreSQL already uses port 5432, stop it or change the host port in compose. The container port stays 5432.
- Check health with `docker compose ps`. If it is not running, read `docker compose logs db`.

DONE WHEN
`db` shows as running, and you can connect to it from the host using the Step 1.3 URL.

Part 2: Backend boots and answers over HTTP and Socket.IO

Goal: the server starts, answers a health check, and accepts a Socket.IO handshake.

Step 2.1: Start the server with the correct target

- Run uvicorn from the backend root with the target `app.main:app`. Keep that name: it refers to the combined Socket.IO and FastAPI app.
- Use `--host 0.0.0.0` so phones on your Wi-Fi can reach the server later (Step 4.5).
- Keep `--reload` while developing.

> **DONE WHEN**
> The server reports it is listening on port 8000 with no errors.

Step 2.2: Verify the HTTP health route

- Open `/health` in a browser.
- Include the router from `app/router/players.py` in the app. It is defined but never included, so its routes are unreachable.

> **DONE WHEN**
> `/health` returns `ok`, and the players route appears in the API docs at `/docs`.

Step 2.3: Verify the Socket.IO endpoint over plain HTTP

- The handshake lives at `/socket.io/`. Request it in a browser with the protocol and transport parameters for polling (`EIO=4` and `transport=polling`).
- A correct response starts with a digit and includes a session id. "Not found" means the mount is wrong.

> **DONE WHEN**
> The handshake URL returns a Socket.IO open packet containing a session id.

Step 2.4: Replace print statements with logging

- The connect and disconnect handlers use `print`. Switch to the standard `logging` module so output can be filtered by level later.
- Log the session id, the disconnect reason, and the auth key names. Never log auth values.

> **DONE WHEN**
> Connect and disconnect messages appear through the logger at INFO level.

Part 3: Database connection and migrations

Goal: the app reads and writes PostgreSQL, and Alembic manages the schema.

Step 3.1: Add a database health route

- Add a route such as `/health/db` that runs a trivial query through the session from `get_db`.
- When the database is down, return a clear failure status and message, not an unhandled 500.

> **DONE WHEN**
> `/health/db` returns OK while the container runs, and a clear failure after `docker compose stop db`.

Step 3.2: Decide sync or async database access (before writing models)

- The engine is synchronous (`create_engine`), but Socket.IO handlers are `async`.
- **Recommendation:** keep the synchronous engine for now. Run database work from async handlers through a threadpool helper, and keep each call short. Switch to async SQLAlchemy only if you measure a problem.
- Write the decision in one line in your notes so it is not reopened later.

> **DONE WHEN**
> The decision is written down, and you know which helper socket handlers will call for database work.

Step 3.3: Connect Alembic to your settings and metadata

- In `alembic/env.py`, set the target metadata to your declarative base's metadata. It is `None` now, so autogenerate sees no tables.
- Read the database URL from your settings, not from a second copy in `alembic.ini`. One source of truth.
- Import every model module before autogenerate runs, or its tables are silently skipped.

> **DONE WHEN**
> `alembic current` runs without errors on the empty database.

Step 3.4: Create and apply the first migration

- Create a first revision for one small table only. The full schema comes in Part 5.
- Apply it with `alembic upgrade head`, then confirm the table in a SQL client (DBeaver, pgAdmin, or `docker compose exec db psql`).
- Practice one downgrade and upgrade so you know rollback works before you need it.

> **DONE WHEN**
> The table exists, `alembic downgrade -1` removes it, and `alembic upgrade head` recreates it.

Part 4: Frontend connects to backend (first socket handshake)

Goal: the browser opens a Socket.IO connection to the backend, and both sides show it.

Step 4.1: Move the backend address into a frontend env variable

- `src/utils/socket.ts` hardcodes `http://localhost:8000`. Move it to an environment variable.
- Vite exposes only variables starting with `VITE_`. The current `BACKEND_APP_URL` in the frontend `.env` is invisible to the browser, so rename it to `VITE_BACKEND_URL`.
- Add a frontend `.env.example` with the same key and a placeholder.
- Restart the Vite dev server after any `.env` change. Vite reads it only at startup.

DONE WHEN
Changing `VITE_BACKEND_URL` and restarting Vite changes the address the socket uses.

Step 4.2: Decide the allowed origins

- The Socket.IO server allows every origin (`*`). That is fine for the first test. Write down the list you will use: `http://localhost:5173`, and your PC's LAN address on port 5173 for phone testing.
- Restrict the list before deploying (Part 8).

DONE WHEN
The exact origin list is written down, using your PC's current LAN IP.

Step 4.3: Turn the connection back on in the app

- `App.tsx` has the socket disconnected. Re-enable the connect call and the connect, disconnect, and connect-error listeners.
- Make the unmount cleanup call disconnect, so React's development double-mount does not leave two sockets open.

DONE WHEN
The browser console shows "Connected" with a socket id, and the backend logs a connect line for the same id.

Step 4.4: Add a ping event as the first end-to-end test

- Backend: one handler, `client.ping`, that replies with `server.pong` and the server time. Use a Socket.IO acknowledgement so the client receives the reply directly.
- Frontend: send one ping after connecting and log the round-trip time (a temporary log is fine).

DONE WHEN
The console prints a round-trip time and the backend logs the ping. Two-way traffic is proven.

Step 4.5: Test from a phone on the same Wi-Fi

- Run the backend with `--host 0.0.0.0` (Step 2.1), and the Vite server with `--host` so it listens on the LAN.
- Allow ports 8000 and 5173 through Windows Firewall for private networks.
- Set `VITE_BACKEND_URL` to the LAN address. On a phone, `localhost` means the phone itself.
- Add the LAN origin from Step 4.2.

> **DONE WHEN**
> The phone loads the game page and the ping round-trip works.

Step 4.6: Test reconnect behavior

- Stop the backend while the page is open. The frontend should show a disconnected state and retry on its own.
- Start the backend again. The socket should reconnect with a new socket id.

> **DONE WHEN**
> The frontend reconnects after a backend restart, without a page reload.

Part 5: Schema and architecture decisions (your design work)

Goal: decide the data model before writing more events. You design it. This section lists what you must decide so nothing is missed.

Step 5.1: List the entities the game needs

Entity	Why it exists
Room	Code, host, status, settings (category, difficulty, max players), round end time
Player	Display name, guest identity, room membership, team, ready state
Round	Story, round number, phase timestamps, outcome
Scene card	The private prompt for one artist in one round
Drawing event or stroke	Each stroke, undo, and clear (matches the frontend event log)
Submission	An artist's final drawing for a round
Guess and score	Guessers' reconstructions and the points awarded
Story (AI output)	The generated story and scene split, kept for replay and debugging

> **DONE WHEN**
> Each entity has a one-line purpose, and you can say which room it belongs to.

Step 5.2: Decide identity and ids

- Decide how a guest is identified across reconnects. The frontend keeps one id per browser tab in `sessionStorage`. Decide whether the server issues the id on connect, or the client sends its own.
- Decide id types: UUIDs everywhere, or integer keys internally with UUIDs for anything sent to clients.
- Decide how room codes are generated and kept unique, and when old rooms expire.

> **DONE WHEN**
> A written rule says where each id comes from and who may use it.

Step 5.3: Decide the source of truth for live game state

- Decide which state lives in memory (phase, timers, connected players) and which goes to PostgreSQL (rooms, rounds, scores, history).
- Decide whether writes happen immediately or in the background. The design documents prefer background writes, so gameplay never waits on the database.
- Decide what is lost if the server restarts mid-round. A simple rule for development is acceptable.

> **DONE WHEN**
> You can answer: "If the server restarts mid-round, what is lost and what is kept?"

Step 5.4: Decide how stroke data is stored

- Option A: one row per stroke, with points stored as JSON. Simple, and matches the frontend's current format.
- Option B: one row per drawing event (stroke, undo, clear) in an ordered event table. Easiest to replay and debug.
- Decide how long the event log is kept, and whether it is kept after a round ends.

> **DONE WHEN**
> You have chosen A, B, or a combination, and can justify it in two sentences.

Step 5.5: Write the schema diagram

- Draw the tables and their relationships. Mark primary keys, foreign keys, and unique constraints.
- Check each query you expect: "get a room with its players", "get the current round's scene cards", "list a room's drawing events in order". The schema must answer each one.

> **DONE WHEN**
> Every expected query can be answered from the diagram without guessing.

Step 5.6: Decide the team and match model

- Separate **team** (Team A and Team B, fixed for the whole match) from **role** (artist or guesser, changes every round). The current frontend stores only a role per player, so this is a schema change.
- Decide the round count. The design documents use 6 rounds by default, so each team draws three times. The create-room form has no round count yet.
- Decide the rule for uneven teams and for the minimum scene count (the documents raise 2 artists as a case to handle).

> **DONE WHEN**
> For a 5-player room, you can state who is on each team and who draws in rounds 1 and 2.

Part 6: Socket events, one at a time

Goal: build the game flow over Socket.IO in small steps. An event is finished only when the frontend can trigger it and see the result.

Rules for every event: (1) write its payload in the protocol doc first; (2) validate every field on the server and reject bad input with an error reply; (3) use an acknowledgement for requests that need an answer; (4) broadcast state changes to the room; (5) test from the browser before starting the next event.

Protocol document (do this first)

Create `docs/websocket-protocol.md` in the backend. For each event, list: name, direction (client to server, or server to clients), payload fields with types, the acknowledgement reply, and the error codes it can return. Use the envelope the frontend already sends for drawing events: `version`, `eventId`, `roomCode`, `playerId`, `sequence`, `sentAt`, plus the type-specific fields.

Event order

#	Event (client to server)	Result sent to clients	Test from the frontend
6.1	`room.create` (name, settings)	Acknowledgement with room code; `room.state` to the creator	Create a lobby; a code appears
6.2	`room.join` (code, name)	Acknowledgement; `room.state` to everyone in the room	Join from a second browser; both lobbies update
6.3	`room.leave`, plus automatic leave on disconnect	`room.state`; host transfers if the host leaves	Close one tab; the other shows the player leave
6.4	`player.ready` (toggle)	`room.state`	Ready toggles on both screens
6.5	`player.setTeam` (artists or guessers)	`room.state`; rejected while a round runs	Switch teams in the lobby

6.6	`game.start` (host only)	`room.state` to all; `scene.assigned` to each artist only	Start a game; each artist sees only their own card
6.7	`drawing.stroke`, `drawing.undo`, `drawing.clear` (artists only)	Same event broadcast to the room after validation	Draw in one browser; a second browser sees it live
6.8	`drawing.submit`	Artist locked in; `room.state`; round ends when all artists submit or time runs out	Submit from each artist; the gallery opens
6.9	`game.reset` (host only)	`room.state`; back to the lobby	"Play another round" returns everyone to the lobby

Checks to add to each event

- **Authorization:** the sender is in the room and has the right role (host, artist).
- **Phase:** the event is allowed in the current phase (for example, no strokes outside the drawing phase).
- **Order:** `sequence` increases per player; duplicates are dropped by `eventId`.
- **Size:** reject strokes with too many points or too large a payload.

Mapping to the frontend (so Part 7 is simple)

Frontend function (roomService.ts)	Socket event
`createRoom`	`room.create`
`joinRoom`	`room.join`
`getRoom` / `subscribeToRooms`	`room.state`, pushed by the server (no polling)
`setReady`	`player.ready`
`setTeam`	`player.setTeam`
`startGame`	`game.start`
`submitDrawing`	`drawing.submit`
`finishRound`	Removed from the client; the server ends the round
`resetLobby`	`game.reset`
`leaveRoom`	`room.leave`
`publishDrawingEvent`	`drawing.stroke`, `drawing.undo`, `drawing.clear`

Part 7: Replace the frontend's local stand-in with the real socket client

Goal: the screens keep working, but the data now comes from the server.

Step 7.1: Create one socket client module

- Create a single module that owns the socket. It exposes `connect`, `disconnect`, and promise-returning `emit` helpers, plus a way to subscribe to incoming events.
- Only this module imports `socket.io-client`. Everything else calls it.

DONE WHEN
Only one file in `src/` imports `socket.io-client`.

Step 7.2: Swap the room functions one at a time

- Keep each function's name and signature in `roomService.ts`. Change only the body, so it calls the socket client.
- Swap in the order of Part 6. After each swap, test the screen that uses it before moving on.
- Map server error replies to the same messages the screens show today.

DONE WHEN
Each swapped function works in two browsers before you move to the next.

Step 7.3: Swap the drawing log to the socket

- Keep the local event log as a replay cache, but send each event to the server as it happens.
- Subscribe to incoming `drawing.*` events and feed them into the same replay code.
- Handle events that arrive out of order or twice, using `eventId` and `sequence`.

DONE WHEN
Strokes drawn on one device appear on another in real time, and reloading restores the current canvases.

Step 7.4: Remove the client-side round end logic

- In `DrawingScreen.tsx`, remove the timers that call `finishRound`. The server decides when a round ends and pushes the new state.
- Keep only the countdown display, driven by the server's deadline.

DONE WHEN
Both browsers move to the gallery at the same moment, as decided by the server.

Step 7.5: Decide what happens to the local stand-in (optional)

- Keep it behind a flag if you want offline development, or delete it once the socket version is stable.

DONE WHEN
You have decided: keep behind a flag, or delete.

Part 8: AI story and scene generation

Goal: before each round, the server asks an AI for a story and one scene card per artist, validates the result, and stores it. The AI never sees players, scores, or drawings, and it never runs while people are drawing.

Step 8.1: Define the AI output contract

- Each round needs one structured response: the story (3 to 8 sentences, self-contained, funny, family-safe), one scene card per artist, and for each scene a short list of key story beats.
- The key beats are what scoring (Part 9) matches guesses against. Ask for them now, so scoring never has to guess what the story was about.
- Parse the response into a fixed structure. Reject any response that does not match it, and never pass raw model text to clients.

DONE WHEN
You have a written example response for 3 artists, and the server rejects a malformed one.

Step 8.2: Choose a provider and keep it swappable

- Put the provider behind one interface, for example `generate_round(category, difficulty, artist_count)`. Build one concrete provider first. The design documents require the rest of the code to not depend on which one it is.
- Keys live only in the backend's `.env` (provider name, API key, model name). Add the names to `.env.example`. Never send them to the frontend.
- A small, fast model is usually enough for story and scene generation. Upgrade only if output quality is poor.

DONE WHEN
A one-off test calls the provider with a fixed category and gets back a parsed response.

Step 8.3: Set the scene count rules

- The number of scenes equals the number of artists. Decide a minimum (the phase plan suggests about 3, so a 2-artist round asks for a longer story) and a maximum (about 8, by splitting large teams into parallel sub-stories).
- Write the rule in your notes, and make the limits constants rather than numbers scattered through the code.

DONE WHEN
The rule is written, and a test shows it works for 2, 3, and 8 artists.

Step 8.4: Write the scene validator (in code, not in the prompt)

- Do not trust the model to follow the scene rules. Check them in code:
  - The scene is a visual instruction (for example, it starts with "Draw" or describes a subject doing something).
  - No causal or sequencing words such as because, before, after, then, however, accidentally, suddenly, later.
  - Not a single word or abstract connector: enforce a minimum word count.
  - No full spoilers: a scene must not repeat the story's final beat.
  - Balance: flag a round where one scene carries far more story detail (beat count or word count) than the others.
- If a check fails, regenerate. Set a retry limit (2 or 3 attempts).

> **DONE WHEN**
> The validator rejects the bad examples from the design document and accepts the good ones.

Step 8.5: Generate before drawing, not during it

- When the host starts the game, move to a GENERATING state and broadcast the room state without any story content.
- Generate in the background so the socket loop keeps responding. Then move to DISTRIBUTING (send each artist their private `scene.assigned`) and then DRAWING, with the server-set deadline.
- Decide whether to pre-generate the next round during the current round's guessing and scoring. That removes the wait between rounds. Document the decision.

> **DONE WHEN**
> Starting a game shows "Generating story" to everyone, and the drawing timer starts only after the content is ready.

Step 8.6: Plan for AI failures

- On a timeout or error, retry with a short backoff (2 attempts).
- If it still fails, use a fallback round from a local file of prewritten stories, in the same structure. The game must keep moving.
- Flag fallback rounds in the room state so the host knows, and log every fallback.

> **DONE WHEN**
> With a wrong API key or no network, a game still reaches DRAWING using fallback content.

Step 8.7: Store the round content

- Save the story, the scene cards, the key beats, the provider, the model name, the prompt version, the time, and whether it was a fallback. You need this for replay and debugging.
- The story stays secret until the reveal (Part 9). Store it, but never send it before then.

DONE WHEN
A finished round's story and scenes can be read back from the database.

Step 8.8: Update the frontend

- Show a "Generating story" state between the lobby and the drawing screen.
- Take scene cards from `scene.assigned`, not the placeholder list in `src/constants/game.ts`. Remove that list once nothing uses it.

DONE WHEN
Each round shows AI-generated scene cards, and they differ between rounds.

Part 9: Guessing, reveal and scoring

Goal: after drawing, guessers write their reconstruction, the story is revealed, and both teams are scored by the server.

Step 9.1: Decide the scoring rules first

The two design documents disagree. Version 0.1 scores semantic similarity, speed, scene clarity, and a balance bonus. Version 0.2 scores artists by community votes. The phase plan says the first version uses keyword matching only. Pick one set before you write any scoring code.

Recommended first version:

Team	Points for	How it is measured (first version)
Guessers	Understanding the story	Share of the story's key beats found in the team's guess, 0 to 100
Guessers	Speed (optional)	Small bonus for the first team to submit
Artists	Communicating the story	The opposing team's guess score (same measure), 0 to 100
Artists	Self-reconstruction (optional)	Artists guess the whole story too, since they saw only their scene
Artists	Helpful-panel votes (later)	Bonus per vote, from Step 9.6

- Make every weight a named constant, so you can tune it without touching logic.

DONE WHEN
You have written the final weights and chosen whether artists are scored by the opposing guess (recommended) or by votes.

Step 9.2: Accept guesses

- Add the `guess.submit` event (text only). Only guessers may send it, and only during the GUESSING phase.
- Validate the text: not empty, a length limit, and rejected once the deadline passes.
- Decide whether each guesser submits their own guess and the team score uses the best one, or whether each team submits one guess (for example, the latest before the deadline). Write the rule down.

> **DONE WHEN**
> A valid guess is stored, empty and late guesses are rejected, and a duplicate is handled by your rule.

Step 9.3: End guessing and reveal the round

- End the phase when the timer runs out or every guesser has submitted. Move to REVEAL and broadcast the story, the scene cards, the scene order, and all drawings.
- Decide who may see what in each phase. Use this as the rule your server enforces:

Phase	Artists see	Guessers see
GENERATING / DISTRIBUTING	Their own scene card only	Nothing about the story
DRAWING	Own card, teammates' drawings as they arrive	Artists' drawings as they arrive (no cards)
GUESSING	Drawings; their own guess	Drawings; their team's guess
REVEAL onward	Everything	Everything

> **DONE WHEN**
> All clients get the reveal at the same moment, and an inspection of socket traffic shows the story was never sent earlier.

Step 9.4: Score guesses on the server

- Normalize text (lowercase, remove punctuation, match simple word forms). Count how many of the story's key beats appear in the guess, then convert to a 0 to 100 score.
- Store each guess's score and which beats matched. The reveal screen can then show why a guess scored what it did.
- Keep this as a pure function that takes text and beats, so you can test it.

> **DONE WHEN**
> A unit test with a fixed story and guess returns the expected score and matched beats.

Step 9.5: Score the drawings and the rest of the round

- Compute artist points from the opposing guess score (Step 9.1), plus any speed or self-reconstruction bonuses you chose.
- Compute all points in one pure function that returns a per-player and per-team breakdown.

> **DONE WHEN**
> Unit tests with fixed inputs produce the expected breakdown for both teams.

Step 9.6: Add community votes (optional, after the first version works)

- After the reveal, each player votes once for the most helpful drawing, and never for their own.
- The server counts votes and adds a bonus to each artist per vote. Fun awards do not affect points.

> **DONE WHEN**
> One vote per player is enforced, self-votes are rejected, and vote points appear in the round score.

Step 9.7: Broadcast results and update the frontend

- Send `round.reveal` with the reveal content, then `round.scored` with the breakdown.
- Frontend: a reveal screen showing the comic strip in story order (drawing plus sentence), the guesses with matched beats, and the round score per team.

> **DONE WHEN**
> Every client sees the same reveal and the same score breakdown.

Part 10: Round switching and match end

Goal: after scoring, roles swap, the next round starts, and the match ends with a winner.

Step 10.1: Write the round state machine

- Full loop: LOBBY, GENERATING, DISTRIBUTING, DRAWING, GUESSING, REVEAL, SCORING, RESULTS, then either the next round's GENERATING or FINISHED.
- Write the allowed transitions as a table in the protocol doc. Any other transition is rejected.

> **DONE WHEN**
> A unit test tries an illegal move (for example, DRAWING straight to SCORING) and it is rejected.

Step 10.2: Swap roles between rounds

- Each round, one team draws and the other guesses. Next round they switch. Compute roles from the team and the round number, rather than storing them separately, to avoid drift.
- Decide the rule for uneven teams, for example rotating one player per round so everyone draws at least once.

> **DONE WHEN**
> A written 4-player table shows who draws in each round, and each player draws equally often.

Step 10.3: Set the round count and match length

- Default is 6 rounds, so each team draws 3 times. The count must be even so both teams draw equally. Enforce this on the server.
- Add the round count to the create-room settings and the frontend form.
- Optional: escalate difficulty across rounds (Easy, then Medium, then Hard), as the design document suggests.

> **DONE WHEN**
> A 6-round match runs from start to finish, and the round counter shows on every screen.

Step 10.4: Decide the winner and tie-breaks

- At FINISHED, the winner is the team with the higher cumulative score. Show the full leaderboard.
- Tie-break options from the design document: a sudden-death round (one short scene each), then total scene clarity bonus, then co-winners in casual mode.
- Recommended: co-winners first (simplest), and add sudden death later.

> **DONE WHEN**
> The final screen shows both teams with a winner or co-winners, and a forced tie triggers your chosen rule.

Step 10.5: Allow a second match in the same room

- When the host chooses "play again", start a new match with the same players and zero scores. Keep the first match's history in the database.
- This builds on the `game.reset` event from Part 6.

> **DONE WHEN**
> The second match starts at zero, and the first match is still in the database.

Step 10.6: Update the frontend for teams and roles

- The lobby shows Team A and Team B, not only artists and guessers. Each round, show the player's role (Artist or Guesser) and the round counter.
- Add a team scoreboard and a final leaderboard screen.

> **DONE WHEN**
> A full match plays in four browsers, and every screen shows the correct team and role for the current round.

New events for Parts 8 to 10

Event	Direction	Purpose
`round.generating`	Server to clients	Content is being prepared (no story included)
`scene.assigned`	Server to one artist	Private scene card (already in Part 6)
`guess.submit`	Guesser to server	Text reconstruction of the story
`round.reveal`	Server to clients	Story, scene order, and all drawings
`round.scored`	Server to clients	Per-player and per-team score breakdown
`match.state`	Server to clients	Round number, team scores, roles for the round
`match.finished`	Server to clients	Final leaderboard and winner or co-winners

Part 11: Later phases

Start these only after Parts 1–10 work end to end. Each one gets its own plan later.

Phase	What it adds	Start when
Reliability	Reconnect and resume using `sequence`, heartbeats, disconnect rules, idle room cleanup	Two-device testing shows drops
Persistence and history	Save rounds and scores to PostgreSQL in the background	Scoring works
Deployment	Restrict origins, secure WebSocket (wss://) behind a reverse proxy, production env handling, secrets	Gameplay is stable

Appendix A: Issues found in the current files

#	Where	Issue	Fixed in
1	`docker-compose.yml`	Backend service has no `DATABASE_URL`	Step 1.4
2	`app/db/database.py`	Reads `.env` relative to the current folder; old `class Config` style	Step 1.5

3	`Dockerfile`	Copies only `./app`; Alembic files sit outside it	Steps 1.4 and 3.3 (decide whether migrations run in the container)
4	`requirements.txt`	Appears to be saved as UTF-16	Step 1.6
5	`app/main.py`	The players router is never included	Step 2.2
6	`alembic/env.py`	`target_metadata = None`, so autogenerate sees no tables	Step 3.3
7	`app/util/socketio.py`	Origins set to `*`; handlers use `print`	Steps 2.4 and 4.2
8	Frontend `src/utils/socket.ts`	Backend URL hardcoded to `localhost:8000`	Step 4.1
9	Frontend `.env`	`BACKEND_APP_URL` is not exposed to the browser (needs a `VITE_` prefix)	Step 4.1
10	Frontend `App.tsx`	Socket connection is not active	Step 4.3
11	Backend git status	Many older files show as deleted and uncommitted	Decide before Part 1 is complete: commit the new layout or restore the old files
12	Design documents	Version 0.1 and version 0.2 of the GDD give different scoring rules	Step 9.1 (pick one set)
13	Frontend `roomService.ts`	Stores one role per player; no team concept (Team A and Team B) and no round count	Steps 5.6 and 10.3
14	Frontend `src/constants/game.ts`	Placeholder scene cards are used instead of AI-generated ones	Step 8.8

Appendix B: Checkpoint summary

Checkpoint	You have reached it when
C1: Environment	Steps 1.1–1.7 pass; a fresh venv and the Docker database both work
C2: Backend up	Steps 2.1–2.4 pass; `/health` and the Socket.IO handshake respond
C3: Database up	Steps 3.1–3.4 pass; the first migration applies and rolls back
C4: Frontend connected	Steps 4.1–4.6 pass; ping works from a laptop and a phone
C5: Design settled	Steps 5.1–5.5 are written down, and the diagram answers every expected query
C6: Lobby over sockets	Events 6.1–6.5 pass in two browsers
C7: Round over sockets	Events 6.6–6.9 pass; the server ends the round
C8: Frontend fully live	Part 7 passes; the local stand-in is no longer needed for the main flow

C9: AI content	Steps 8.1–8.8 pass; each round has validated AI scenes, and a fallback works when the provider fails
C10: Scoring	Steps 9.1–9.7 pass; guesses and drawings are scored by the server and shown at reveal
C11: Full match	Steps 10.1–10.6 pass; a 6-round match with role swaps ends with a leaderboard